Tecnologia em Sistemas para Internet

DISCIPLINA: ESTRUTURAS DE DADOS

LISTAS LIGADAS (ENCADEADAS)

- Prof. Alexandre Ponce de Oliveira

alexandreponceo@gmail.com

Aulas 29 e 30

Lins/SP

Listas Ligadas (Encadeadas)

Listas são estruturas (complexas) de dados onde cada elemento contém uma ligação (endereço) para o próximo elemento da lista.

Assim, uma lista pode ser acessada de maneira randômica, podendo ser retirados, acrescidos ou inseridos elementos no meio da lista dinamicamente.

Flexibilidade maior em relação a vetores

Tipos específicos de listas:

- Fila: primeiro a entrar, primeiro a sair;
- Pilha: último a entrar, primeiro a sair.

Listas Ligadas (Encadeadas)

Uma lista é simplesmente ligada, se o ponteiro anterior em cada elemento é omitido.

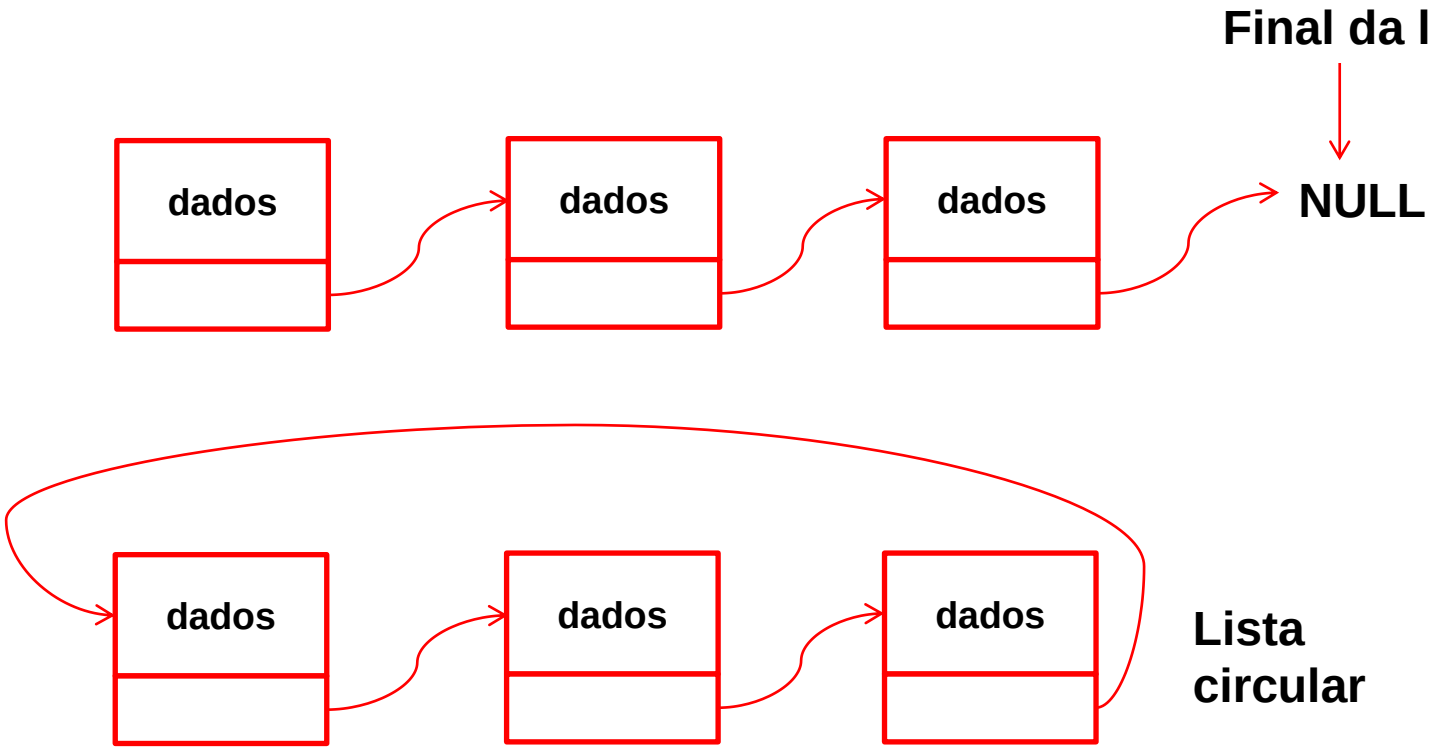
Uma lista é ordenada, se o elemento mínimo é o início da lista e o elemento máximo é o fim.

Uma lista é não ordenada, se os elementos podem aparecer em qualquer ordem.

Em uma lista circular (anel de elementos), o ponteiro próximo do fim da lista aponta para o início da lista.

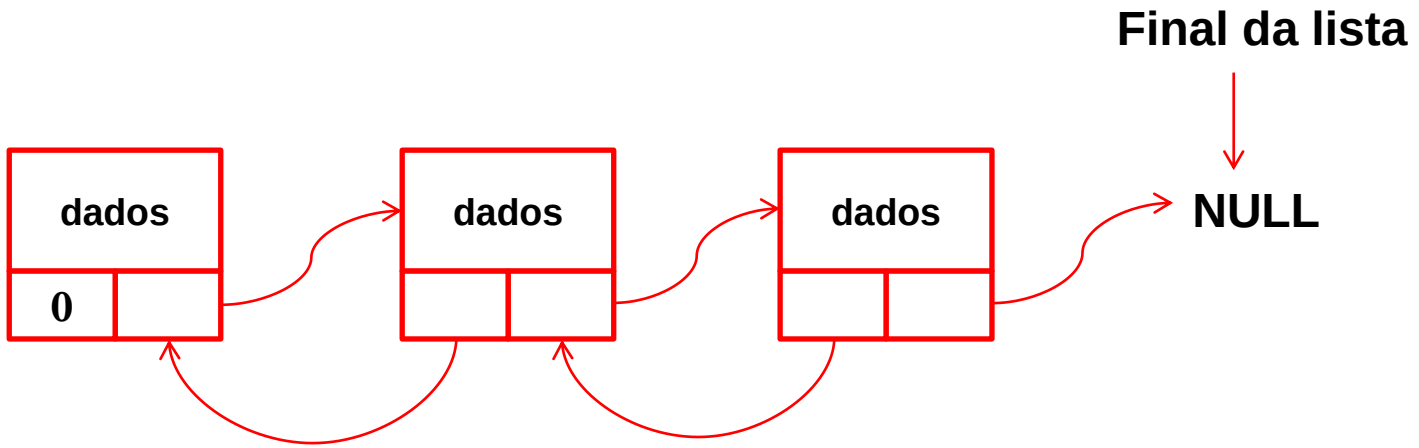
Listas Ligadas Simples

Um lista ligada simples pode ser representada por uma estrutura que contém campos de informação e o endereço (ponteiro) do próximo elemento:



Listas Duplamente Ligadas

Um lista duplamente ligada pode ser representada por uma estrutura que contém campos de informação e os endereços (ponteiros) do próximo elemento e do elemento precedente:

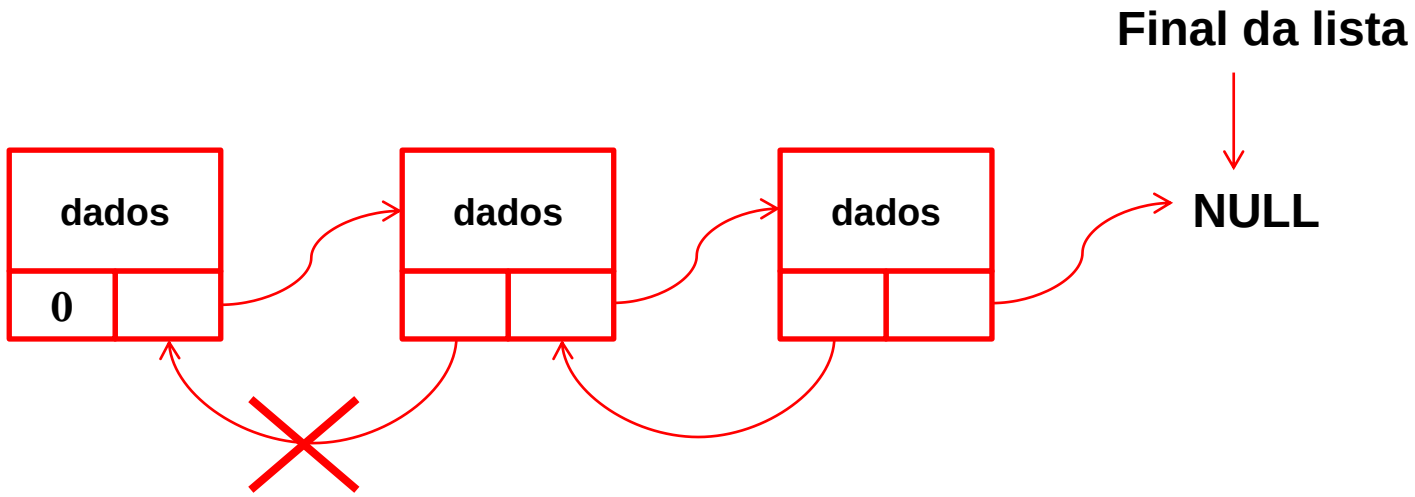


Listas Duplamente Ligadas

Vantagens:

- Pode ser lida nas duas direções.

- Caso um dos apontamentos falhar, a lista pode ser reconstruída com a utilização do outro apontamento.



Estrutura das Listas Ligadas

Lista Ligada Simples

```
struct No{
        int valor;
        struct No *prox;
};
typedef struct No Lista;
```

Lista Duplamente Ligada

```
struct No{
        int valor;
        struct No *prox;
        struct No *ant;
};
typedef struct No Lista;
```

Listas Ligadas

As operações para manipulação de uma lista são:

- Inserir elementos na lista;
- Retirar elementos na lista;
- Buscar elementos na lista;
- Criar uma lista;
- Destruir uma lista;
- Ordenar uma lista;
- Intercalar duas listas;
- Concatenar duas listas;
- Dividir uma lista em duas;
- Copiar uma lista em outra.

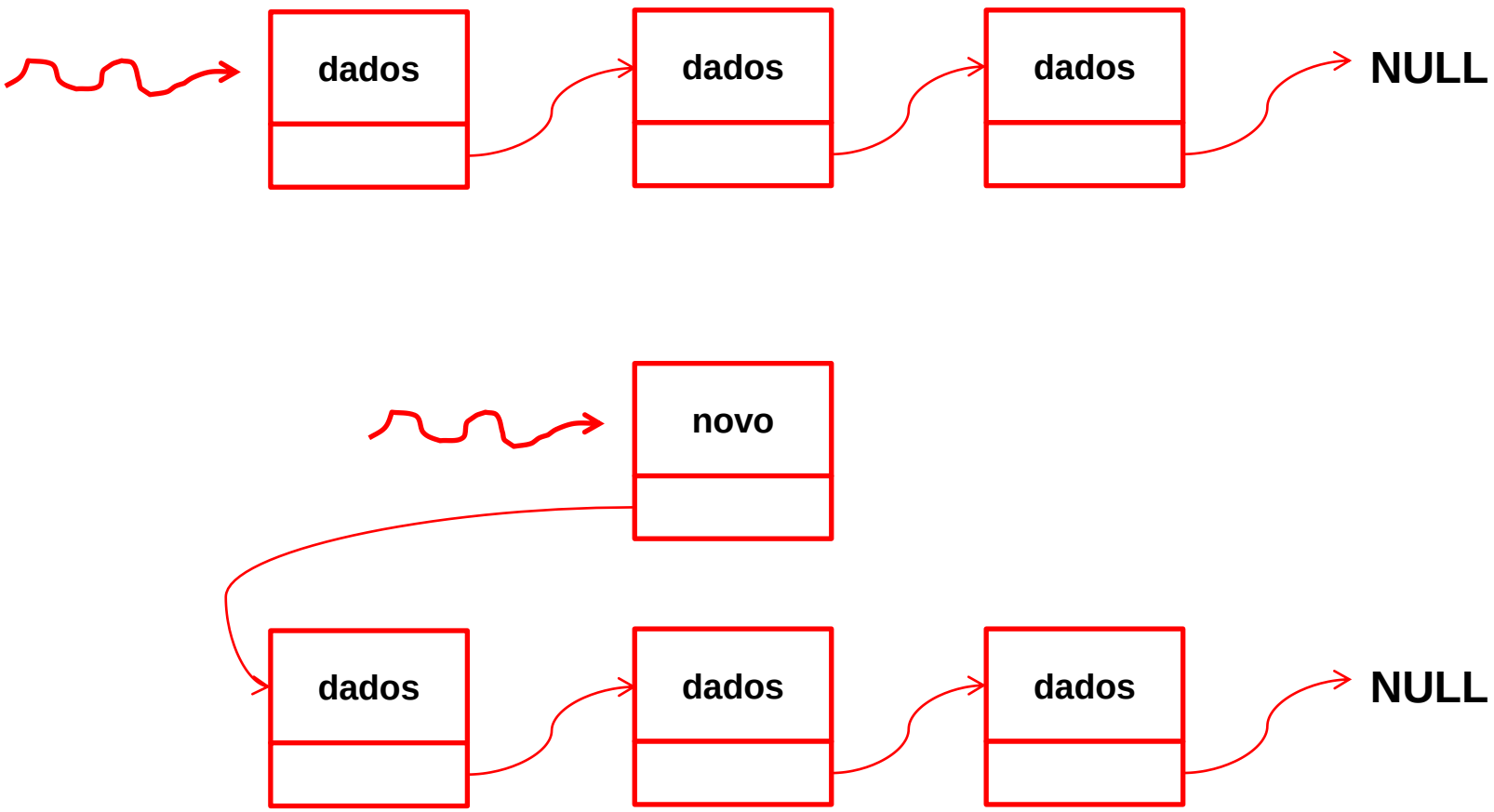
Listas Ligadas

A operação de inserção de elementos pode ser realizada:

- No começo de uma lista;
- No final de uma lista;
- No meio de uma lista.

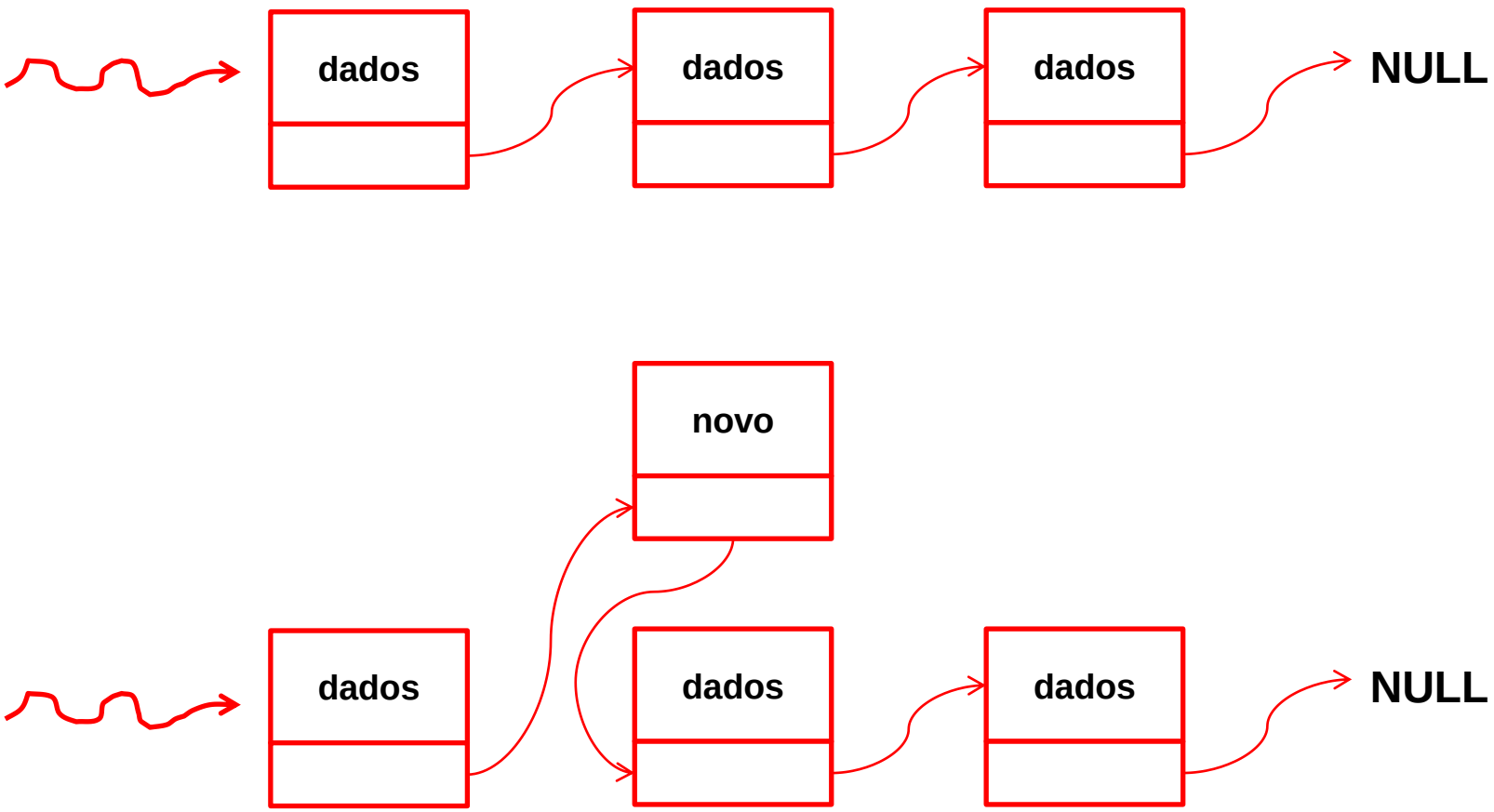
Listas Ligadas

Operação de inserção no começo de uma lista:



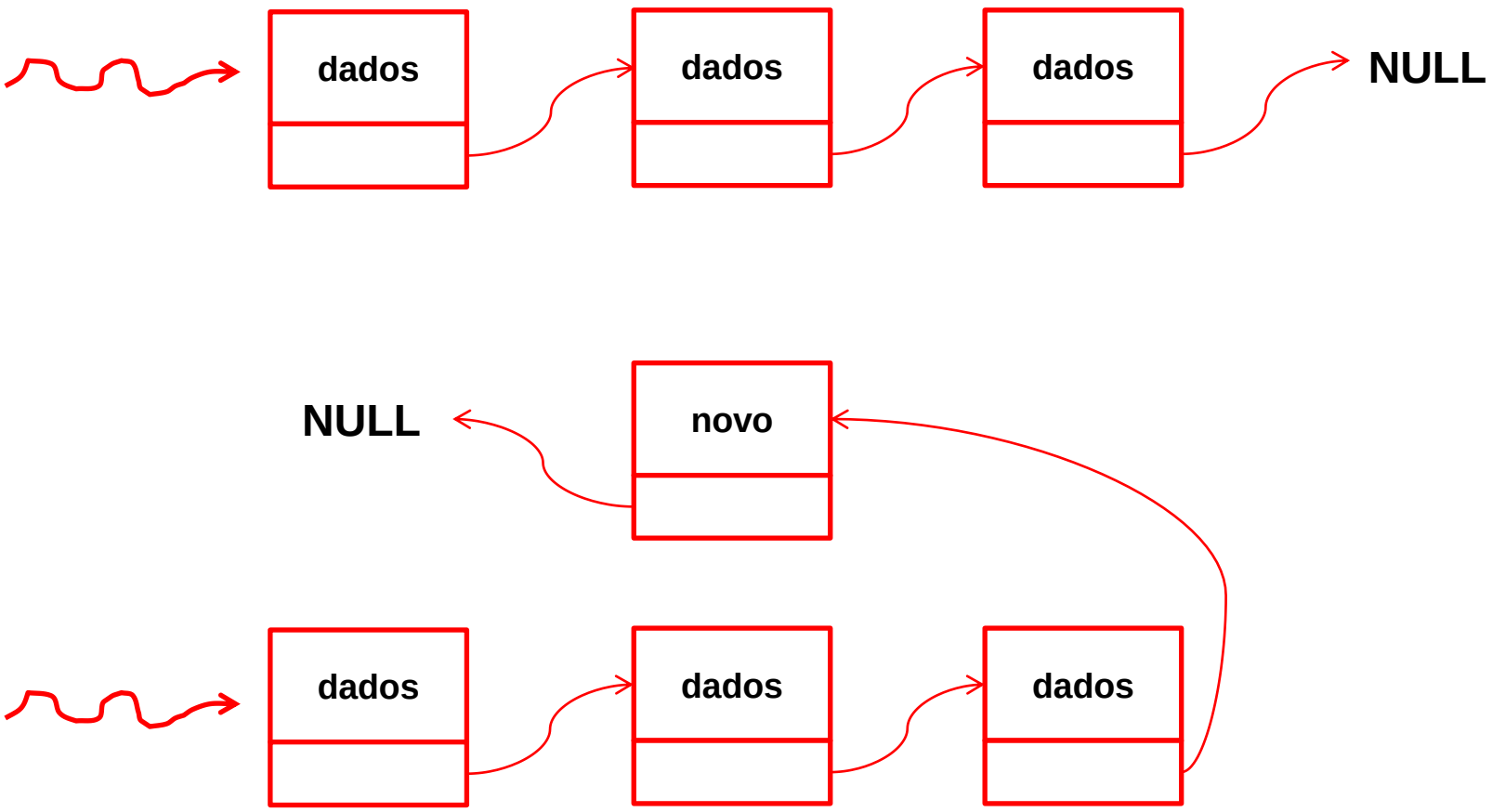
Listas Ligadas

Operação de inserção no meio de uma lista:



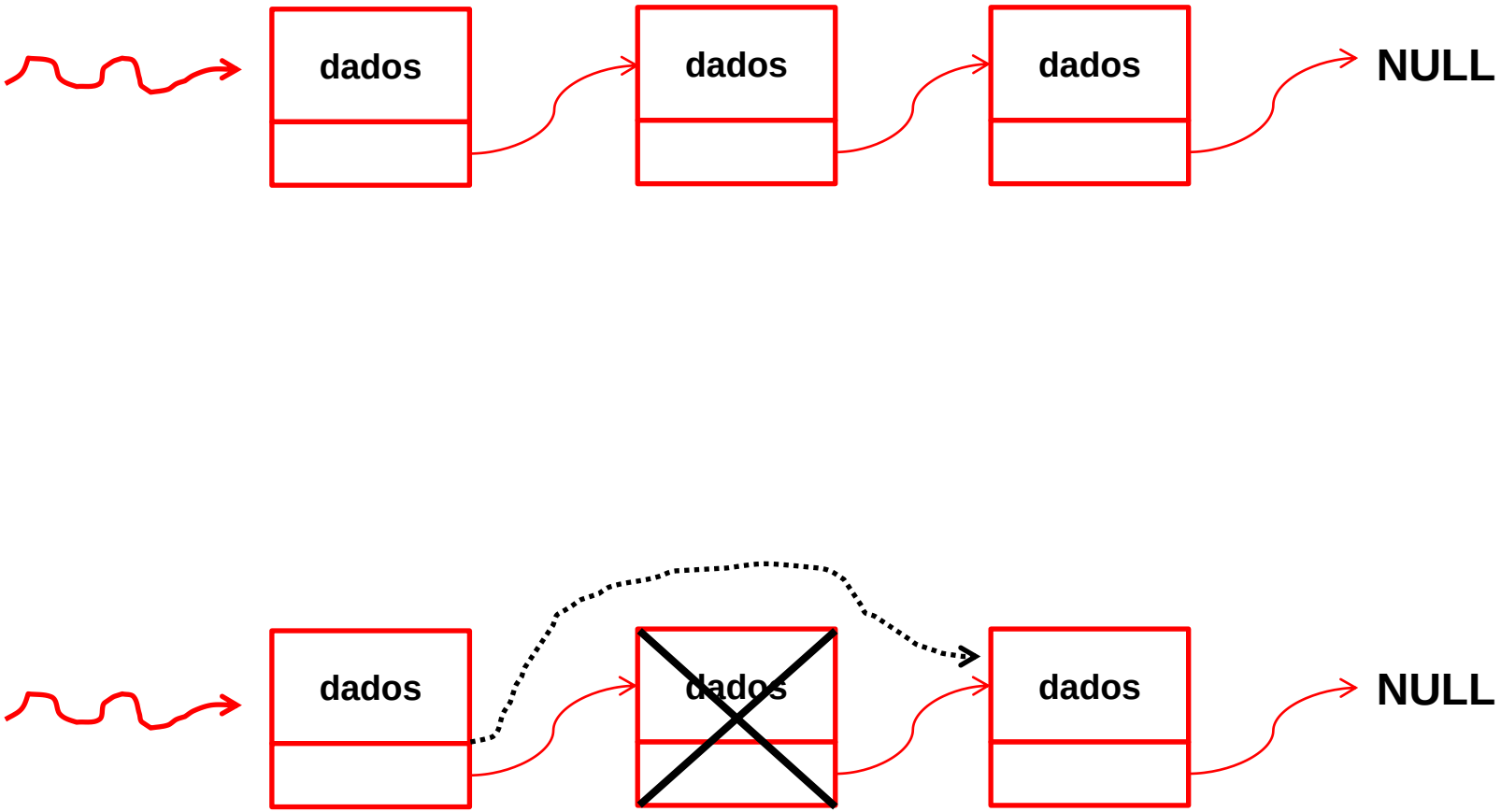
Listas Ligadas

Operação de inserção no final de uma lista:



Listas Ligadas

Operação de exclusão de um elemento do meio da lista:



Criação da estrutura de uma lista ligada simples

```
#include <stdio.h>
#include <stdlib.h>
struct No {
	int valor;
	struct No *prox;
};
typedef struct No lista;

int main(void)
{	lista *cartas = (lista *) malloc(sizeof(lista));
	if(!cartas){
		printf("Impossivel alocar memoria\n");
		exit(1);
	 }
	else
	    printf("Memoria alocada\n");
	free(cartas);
	return 0;
}
```

Inicialização de uma lista ligada simples

//Função

```
void inicializa (lista *cartas)
{
      cartas->prox = NULL;
      printf("Lista cartas inicializada\n");
}
```

//Código na função main()

```
      inicializa(cartas);
```

Checa se a lista está vazia

//Função

```
int checa_lista(lista *cartas)
{
	if(cartas->prox == NULL)
		return 1;
	else
		return 0;
}
```

//Código na função main()

```
	i = checa_lista(cartas);
	if(i == 1)
	    printf("Lista vazia\n");
	else if (i == 0)
	    printf("Lista não vazia\n");
```

Adiciona um elemento no final da lista

```
//Função
void adiciona_fim(lista *cartas)
{  lista *novo = (lista *) malloc(sizeof(lista));
   if(!novo)
   {   printf("Impossivel alocar memoria\n");
       exit(1);
   }
   printf("Digite uma carta para inserir no baralho: ");
   scanf("%d", &novo->valor);
   novo->prox = NULL;
   if(checa_lista(cartas))
       cartas->prox = novo;
   else
   {  lista *aux = cartas->prox;
       while(aux->prox != NULL)
              aux = aux->prox;
       aux->prox = novo;
       }
}
```

```
//Código na função main()
adiciona_fim(cartas);
```

Mostra os elementos da lista

//Função

```
void mostra_lista(lista *cartas)
{
      if(checa_lista(cartas))
      {
          printf("Lista vazia\n");
             return ;
      }
      lista *aux = cartas->prox;
      printf("Cartas -> ");
      while( aux != NULL){
             printf("%d -> ", aux->valor);
             aux = aux->prox;
      }
      printf("NULL \n\n");
}
```

//Código na função main()

```
      mostra_lista(cartas);
```

Adiciona um elemento no início da lista

//Função

```
void adiciona_inicio(lista *cartas)
{
      lista *novo = (lista *) malloc(sizeof(lista));
      if(!novo)
      {
            printf("Impossivel alocar memoria\n");
            exit(1);
    }
      printf("Digite uma carta para inserir no baralho: ");
      scanf("%d", &novo->valor);
      lista *aux = cartas->prox;
      cartas->prox = novo;
      novo->prox = aux;
}
```

//Código na função main()

```
      adiciona_inicio(cartas);
```

Remove um elemento do início da lista

//Função

```
void remove_inicio (lista *cartas)
{
	lista *aux = cartas->prox;
	printf("Removendo o primeiro elemento do baralho \n");
	cartas->prox = cartas->prox -> prox;
	free(aux);
}
```

//Código na função main()

```
	remove_inicio(cartas);
```

Remove um elemento do final da lista

//Função

```
void remove_final (lista *cartas)
{
	while (cartas->prox->prox != NULL)
	{
		cartas = cartas->prox ;
	}
	free(cartas->prox);
	cartas->prox = NULL;
	printf("Removendo o ultimo elemento do baralho \n");
}
```

//Código na função main()

```
	remove_final(cartas);
```

Limpa a lista

```
//Função
void limpa_lista(lista *cartas)
{       if(!checa_lista(cartas))
        {       lista *aux,*atual,*head;
                head = cartas;
                atual = cartas->prox;
                while(atual->prox != NULL)
                {       aux = atual->prox;
                        free(atual);
                        atual = aux;
                }
                if (atual->prox == NULL)
                {   head->prox = NULL;
                   free(atual);
                }
                 printf("Lista limpa com sucesso!!! \n");
        }
}
                //Código na função main()
                limpa_lista(cartas);
```

Verifica o tamanho da lista

//Função

```
int tamanho_lista (lista *cartas)
{
	int i = 0;
	while (cartas->prox != NULL)
	{
		cartas = cartas->prox;
		i++;
	}
	return i;
}
```

//Código na função main()

```
	i = tamanho_lista(cartas);
	printf("Tamanho da lista = %d\n", i);
```